

Introduction to Natural Language Processing

HW 4

LLM Tool Calling Agent

Task introduction

- Build an LLM-based agent that, given a user's conversational context and the current task step, selects the correct tool to execute from a provided list of candidate options.
- For each question, your model will read the `full_context` and the `current_step`. You must output a single correct option (e.g., A, B, C...) corresponding to the appropriate tool.
- This assignment only evaluates the tool selection stage — you do not need to generate the final response or actually execute the tool APIs.
- **Requirement :**
 - Submit a report and your source code to E3
 - Upload your submission to Kaggle

Task rules & constraints

- **Model size limit:** Your model must be $\leq 80\text{B}$ total parameters (MoE included — count total params, not active). We judge this by the model's advertised size in its official name: anything labeled 80B or below qualifies, regardless of the exact parameter count. **Open-weight** only (e.g., Qwen3-80B-A3B, Llama-3.1-70B, Gemma-3-27B).
- **Any approach is allowed:** You are free to use any approach you like — the only constraints are **the model size limit and the open-weight requirement**.
- **API usage policy:** Any API used in your retrieval pipeline (embedding, reranking, query rewriting, etc.) must be backed by an **open-weight model within the 80B parameter limit**. **Closed-source APIs such as GPT-5**, Claude are **NOT** allowed.

Dataset

The dataset consists of step-by-step task execution records. For each sample, the model is provided with the historical context of a task and must select the correct tool for the current step.

- HW4_dataset

File	Data_nums	Description
train.jsonl	13587	Labeled samples for training and experimentation
test.jsonl	1181	Unlabeled samples for submission
sample_submission.csv	1181	Example submission file
addition.jsonl	4824	(Additional Resource) Original full-trajectory data with user questions and complete execution plans.

Data Introduction

Each sample in train.jsonl is a JSON object containing:

- id: Unique question ID.
 - full_context: The historical context and a step-by-step breakdown of the user's overall task.
 - current_step: The specific sub-task that needs to be executed right now.
 - options: A dictionary of candidate tools. Each candidate contains the tool's name, description, arguments, and expected results.
 - answer: The ground-truth correct tool option (e.g., "A", "B", "C").
-
- **Note:** test.jsonl shares the exact same schema but excludes the answer field.
 - **Note:** the **options** pool in the test set is much larger (containing options from A to H) , whereas the training set typically provides fewer options (e.g., A to D).

Data Introduction(cont.)

Additional Resource:dev.jsonl

Each sample in dev.jsonl is a JSON object containing:

- question: The user's original natural language query.
 - plan: An array showing the complete step-by-step breakdown and the expected tool to be called at each step.
 - tools: The complete list of available tools and their schemas.
 - domain: The task category (e.g., Finance, Meeting, Flight).
-
- **Note:** train.jsonl was derived and reformatted from this file. You can use this extra resource to further improve your score.

Kaggle Submission (70%)

- [Kaggle link](#)
Use your [@nycu.edu.tw](#) email address to join the competition.
- Display team name: <student ID>
 - team name error: -5%
- Submission format
 - A CSV file with 2 columns: id, answer.
 - Each row corresponds to one question in test.jsonl.
 - id must match the question ID exactly.
 - answer contains your predicted tool option, which must be a single uppercase letter corresponding to the selected tool.
- There are one simple baseline and one strong baseline, beat them to get the higher score.
- Evaluation metric: Categorization Accuracy

id	answer
0	D
1	B
2	G

Change your team name

Please remember to change the team name to <student ID>, or there will be a deduction of 5 points for HW 4.

2026 INLP HW4

LLM Tool Calling Agent



Settings Overview Data Code Models Discussion Leaderboard Rules Team



Your competition is ready to launch! You've completed 11 of 11 tasks to launch your competition.

[Launch Checklist](#)

Your Team

Everyone that competes in a Competition does so as a team - even if you're competing by yourself. [Learn more.](#)

General

TEAM NAME

Your_Student_ID



Kaggle Submission (70%)

- Public leaderboard is calculated with approximately 30% of the test data, private leaderboard is calculated with the other 70%, so the final standings may be different.
You can only view your private leaderboard score after the competition has ended.
- Your submission is evaluated by **Accuracy**.
- You can submit **at most 5 times each day** and choose 3 of the submissions to be considered for the private leaderboard, or will otherwise default to the best public scoring submissions.

Report Submission

Please answer the following 3 questions in detail:

Q1. Method Description (10%)

- Clearly describe your overall tool-calling pipeline and the main idea behind your method.
- Explain your methodology in detail.

Q2. Comparison of Methods (10%)

- You are required to implement and evaluate both a Prompt-based approach and a Supervised Fine-Tuning (SFT) approach on the same model . Crucially, you must test **BOTH** approaches under the following two configurations:

1.Full-Information Configuration: Provide the complete tool definitions, including tool names, high-level descriptions, and detailed argument schemas.

2.Structural-Only Configuration: Provide **ONLY** the tools' parameter keys and data types. You must completely strip out ALL textual descriptions at all levels (including the tool names /descriptions, parameter-level descriptions within arguments, and return-value descriptions within results).

Within the Prompt-based approach, which configuration performs better? Similarly, within the SFT approach, which configuration performs better? Explain the possible reasons for the differences.

Report Submission

Q3. Overcoming Tool Ambiguity and Misselection (10%)

- In real-world tool calling, many candidate APIs share highly similar names or overlapping descriptions (e.g., `train_ticket_query` vs. `search_train`), leading to frequent misselection errors by the LLM.
- Describe the specific strategies you implemented to resolve this ambiguity and improve the model's ability to distinguish between highly similar tools
- Quantitative Error Statistics: You must provide a statistical breakdown of your model's failure cases. Specifically, calculate the total count and percentage of errors that were caused by this tool ambiguity issue (e.g., provide the proportion of tool-misselection errors relative to your total incorrect predictions).

Please answer the questions in detail to get full credit for each question.

Grading policy

- Kaggle (70%)
 - 30% based on the public leaderboard score and 70% based on the private leaderboard score
 - Basic score :
 - Over strong baseline : 55
 - Over simple bassline : 40
 - Under simple baseline : 25
 - Ranking score:
 $15 - (15/N) * (\text{ranking} - 1)$, N=numbers of people in the interval
- Report (30%)
 - 10% for each quesiton

You will receive 0 points if you do not submit the source code.

E3 Submission

Submit your source code and report to E3 before 6/26(Fri) 23:59, **no late submissions will be accepted !**

Format:

- HW4_<student ID>.zip
 - **Source code** : HW4_<student ID>.py or HW4_<student ID>.ipynb
 - **report** : HW4_<student ID>.pdf

If you have any question about HW 4, please feel free to contact with TA: Lai-Sen Cao through email gp6gp6gp6.cs14@nycu.edu.tw

Have Fun !

